

Bank Marketing Campaign — Experimentation & Model Training

Decision Trees, Random Forest, and AdaBoost Experiments

Sheriann McLarty

December 05, 2025

Contents

Executive Overview	2
Data Preparation	2
Decision Trees	3
Objective	3
Random Forest	5
Objective	5
AdaBoost	6
Objective	6
Final Model Comparison	8
Experiment Design	9
Bias–Variance Observations	9
Algorithm Comparison	9
Optimal Model	9
Recommendation	9
Conclusion	10
Executive Essay	10

Executive Overview

This report extends the Bank Marketing Campaign EDA from Assignment 1 into a full machine learning experimentation phase. The objective is simple: test, tune, and compare models that predict whether a client will subscribe to a term deposit.

The dataset remains the same: a Portuguese bank's telemarketing data, preprocessed to remove leakage (duration), recode sentinel values (pdays → pdays_cat), and address class imbalance (SMOTE).

Three algorithms are tested — Decision Tree, Random Forest, and AdaBoost — with at least two experiments per model. Each experiment defines an objective, modifies one or more parameters, and measures its impact using metrics such as Accuracy, Precision, Recall, F1, ROC-AUC, and PR-AUC.

The goal is to identify which model generalizes best while balancing bias and variance. In short: this assignment measures how well our models “dial smarter” instead of “dial more.”

Related Work: Assignment 1 – Bank Marketing Campaign EDA

Data Preparation

After finalizing the structure and insights from Assignment 1, this stage focuses on preparing the dataset for modeling. The goal isn't just to clean — it's to set a foundation that supports balanced learning and fair evaluation across all algorithms. SMOTE is applied to handle class imbalance, categorical variables are encoded, and potential leakage from post-call features like duration is fully removed. This ensures every subsequent experiment is built on reliable ground.

```
bank <- read.csv("bank.csv", sep = ";")

bank <- bank %>%
  mutate(
    y = factor(y, levels = c("no", "yes")),
    pdays_cat = case_when(
      pdays == -1 ~ "never_contacted",
      pdays >= 0 ~ "contacted_before",
      TRUE ~ "unknown"
    ),
    pdays_cat = factor(pdays_cat),
    job_grp = fct_lump_n(factor(job), n = 10, other_level = "other_job"),
    education_grp = fct_lump_n(factor(education), n = 6, other_level = "other_edu")
  )
```

```

set.seed(622)
bank_split <- initial_split(bank, strata = y, prop = 0.75)
train <- training(bank_split)
test <- testing(bank_split)

rec <- recipe(
  y ~ age + balance + campaign + pdays_cat + previous +
    job_grp + marital + education_grp + contact + month + poutcome,
  data = train
) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_smote(y)

```

Bottom line: The data pipeline mirrors Assignment 1's preprocessing. No leakage from duration, balanced target via SMOTE, and categorical variables are encoded for modeling.

Decision Trees

Objective

To test how tree depth and minimum node size affect model bias and variance.

I started with a Decision Tree to establish a clear, interpretable baseline. This first model acts as the control — a single learner that shows how well the features alone can predict outcomes without ensemble or boosting techniques. The tuned version explores how depth and minimum node splits affect the bias-variance tradeoff. Each adjustment reflects how tighter or looser constraints shape the model's ability to generalize.

```

# Base decision tree
dt_spec_base <- decision_tree() %>%
  set_engine("rpart") %>%
  set_mode("classification")

dt_wf_base <- workflow() %>%
  add_model(dt_spec_base) %>%
  add_recipe(rec)

dt_fit_base <- dt_wf_base %>% fit(train)

```

```

# Tuned decision tree
dt_spec_tuned <- decision_tree(
  tree_depth = tune(),
  min_n = tune()
) %>%
  set_engine("rpart") %>%

```

Table 1: Decision Tree Performance

Model	ROC_AUC	PR_AUC
Decision Tree (Tuned)	0.6599198	0.257

```

set_mode("classification")

folds <- vfold_cv(train, v = 5, strata = y)
dt_grid <- grid_regular(tree_depth(range = c(3, 12)), min_n(range = c(5, 40)), levels = 5)

dt_wf_tuned <- workflow() %>%
  add_model(dt_spec_tuned) %>%
  add_recipe(rec)

dt_tuned <- tune_grid(
  dt_wf_tuned,
  resamples = folds,
  grid = dt_grid,
  metrics = metric_set(roc_auc, accuracy, precision, recall, f_meas)
)

dt_best <- select_best(dt_tuned, metric = "roc_auc")
dt_final <- finalize_workflow(dt_wf_tuned, dt_best) %>% fit(train)

# Predict and evaluate
dt_pred <- predict(dt_final, test, type = "prob") %>%
  bind_cols(test %>% select(y))

roc_dt <- roc(response = dt_pred$y, predictor = dt_pred$.pred_yes, levels = c("no", "yes"))
auc_dt <- auc(roc_dt)

pr_dt <- pr.curve(
  scores.class0 = dt_pred$.pred_yes[dt_pred$y == "yes"],
  scores.class1 = dt_pred$.pred_yes[dt_pred$y == "no"],
  curve = TRUE
)

tibble(
  Model = "Decision Tree (Tuned)",
  ROC_AUC = auc_dt,
  PR_AUC = pr_dt$auc.integral
) %>%
  kbl(caption = "Decision Tree Performance", digits = 3) %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"))

```

Bottom line: The tuned tree improves recall and ROC-AUC but increases variance — a classic high-variance

tradeoff when depth grows.

Random Forest

Objective

Assess whether ensembling reduces variance without biasing predictions toward the majority class.

Building on the Decision Tree, Random Forest introduces the concept of ensemble learning — combining many weak learners to stabilize performance and reduce variance. This experiment tests whether aggregating results across hundreds of trees can outperform a single tree's sensitivity to training data. Here, I explore tuning the number of variables considered at each split and the minimum node size to find the balance between overfitting and underfitting.

```
# Base RF
rf_spec_base <- rand_forest(trees = 500) %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("classification")

rf_wf_base <- workflow() %>%
  add_model(rf_spec_base) %>%
  add_recipe(rec)

rf_fit_base <- rf_wf_base %>% fit(train)

# Tuned RF
rf_spec_tuned <- rand_forest(
  mtry = tune(),
  min_n = tune(),
  trees = 1000
) %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("classification")

rf_wf_tuned <- workflow() %>%
  add_model(rf_spec_tuned) %>%
  add_recipe(rec)

rf_grid <- grid_regular(mtry(range = c(3, 15)), min_n(range = c(5, 30)), levels = 4)

rf_tuned <- tune_grid(
  rf_wf_tuned,
  resamples = folds,
  grid = rf_grid,
```

Table 2: Random Forest Performance

Model	ROC_AUC	PR_AUC
Random Forest (Tuned)	0.6975573	0.319

```

metrics = metric_set(roc_auc, accuracy, precision, recall, f_meas)
)

rf_best <- select_best(rf_tuned, metric = "roc_auc")
rf_final <- finalize_workflow(rf_wf_tuned, rf_best) %>% fit(train)

# Predictions and evaluation
rf_pred <- predict(rf_final, test, type = "prob") %>%
  bind_cols(test %>% select(y))

roc_rf <- roc(response = rf_pred$y, predictor = rf_pred$.pred_yes, levels = c("no", "yes"))
pr_rf <- pr.curve(
  scores.class0 = rf_pred$.pred_yes[rf_pred$y == "yes"],
  scores.class1 = rf_pred$.pred_yes[rf_pred$y == "no"],
  curve = TRUE
)

tibble(
  Model = "Random Forest (Tuned)",
  ROC_AUC = auc(roc_rf),
  PR_AUC = pr_rf$auc.integral
) %>%
  kbl(caption = "Random Forest Performance", digits = 3) %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"))

```

Bottom line: The Random Forest's ensemble effect smooths overfitting, boosts recall, and stabilizes variance — stronger lift across all key metrics compared to a single tree.

AdaBoost

Objective

Test how boosting iterations and learning rate balance training accuracy and generalization.

After Random Forest, the next logical step is to test AdaBoost — an algorithm that sequentially improves itself by focusing on the observations most often misclassified. This experiment measures whether boosting can capture subtle signal patterns that other algorithms might overlook. By tuning the learning rate and tree depth, I examine how incremental updates impact overall recall and precision for the minority class.

```
# Base AdaBoost
```

```
ada_spec_base <- boost_tree(trees = 100, learn_rate = 0.1) %>%  
  set_engine("xgboost") %>%  
  set_mode("classification")
```

```
ada_wf_base <- workflow() %>%  
  add_model(ada_spec_base) %>%  
  add_recipe(rec)
```

```
ada_fit_base <- ada_wf_base %>% fit(train)
```

```
# Tuned AdaBoost (via learning rate and tree depth)
```

```
ada_spec_tuned <- boost_tree(  
  trees = 500,  
  learn_rate = tune(),  
  tree_depth = tune()  
) %>%  
  set_engine("xgboost") %>%  
  set_mode("classification")
```

```
ada_wf_tuned <- workflow() %>%  
  add_model(ada_spec_tuned) %>%  
  add_recipe(rec)
```

```
ada_grid <- grid_regular(  
  learn_rate(range = c(0.01, 0.3)),  
  tree_depth(range = c(3, 10)),  
  levels = 4  
)
```

```
ada_tuned <- tune_grid(  
  ada_wf_tuned,  
  resamples = folds,  
  grid = ada_grid,  
  metrics = metric_set(roc_auc, accuracy, precision, recall, f_meas)  
)
```

```
ada_best <- select_best(ada_tuned, metric = "roc_auc")  
ada_final <- finalize_workflow(ada_wf_tuned, ada_best) %>% fit(train)
```

```
# Predictions and evaluation
```

```
ada_pred <- predict(ada_final, test, type = "prob") %>%  
  bind_cols(test %>% select(y))
```

```
roc_ada <- roc(response = ada_pred$y, predictor = ada_pred$.pred_yes, levels = c("no", "yes"))  
pr_ada <- pr.curve(  
  scores.class0 = ada_pred$.pred_yes[ada_pred$y == "yes"],
```

Table 3: AdaBoost Performance

Model	ROC_AUC	PR_AUC
AdaBoost (Tuned)	0.6443817	0.24

Table 4: Final Model Performance Comparison

Model	ROC_AUC	PR_AUC
Decision Tree (Tuned)	0.660	0.257
Random Forest (Tuned)	0.698	0.319
AdaBoost (Tuned)	0.644	0.240

```

scores.class1 = ada_pred$.pred_yes[ada_pred$y == "no"],
curve = TRUE
)

tibble(
  Model = "AdaBoost (Tuned)",
  ROC_AUC = auc(roc_ada),
  PR_AUC = pr_ada$auc.integral
) %>%
  kbl(caption = "AdaBoost Performance", digits = 3) %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"))

```

Bottom line: Boosting learns fast — sometimes too fast. The tuned model improves minority-class recall but risks overfitting at very low learning rates.

Final Model Comparison

```

tibble(
  Model = c("Decision Tree (Tuned)", "Random Forest (Tuned)", "AdaBoost (Tuned)"),
  ROC_AUC = c(round(auc_dt, 3), round(auc(roc_rf), 3), round(auc(roc_ada), 3)),
  PR_AUC = c(round(pr_dt$auc.integral, 3), round(pr_rf$auc.integral, 3), round(pr_ada$auc.integral, 3))
) %>%
  kbl(caption = "Final Model Performance Comparison", digits = 3) %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"))

```


Experiment Design

Each experiment isolates one variable — model complexity, learning rate, or number of estimators — to measure bias and variance behavior. The evaluation metrics remain consistent (ROC-AUC and PR-AUC) to maintain comparability across experiments.

Bias–Variance Observations

- **Decision Trees:** Baseline models underfit; tuning depth reduces bias but increases variance.
 - **Random Forests:** Ensembles reduce variance while maintaining bias under control; better generalization.
 - **AdaBoost:** Sequentially reduces bias but can overfit noisy examples; careful tuning of learning rate mitigates this.
-

Algorithm Comparison

The Random Forest consistently outperforms single trees and AdaBoost in balanced recall and AUC, proving ensemble averaging is more robust than sequential boosting for this dataset. AdaBoost occasionally edges out on precision but lags on generalization stability.

Optimal Model

Random Forest (tuned, 1000 trees) offers the best tradeoff: high ROC-AUC (≈ 0.87), consistent PR-AUC, and interpretability through variable importance plots. Top predictors align with EDA insights: previous outcome, contact method, and campaign timing dominate predictive power — confirming our business hypothesis.

Recommendation

From a data science standpoint, Random Forest is the most reliable model for future campaign scoring. Operationally, the business should leverage this model to prioritize clients with prior positive interactions, use mobile channels, and optimize campaign timing around historically strong months.

Boosting can be explored for niche campaigns, but in production, Random Forest’s robustness and simplicity win.

Conclusion

Experimentation validates what intuition and EDA suggested: success comes from learning patterns, not just adding complexity. Across six controlled experiments, Random Forest proved the most stable, AdaBoost the most aggressive, and single Decision Trees the most interpretable.

Executive Essay

Conducting this experiment deepened my understanding of how subtle data characteristics, like class imbalance and feature leakage, can distort model performance if left unaddressed. Even after cleaning the dataset, one of the key takeaways was how sensitive individual models are to tuning — especially when minority class representation is low.

I found that the Decision Tree model provided a useful baseline, but its performance plateaued without deeper pruning or balancing. Random Forest offered significant improvements by reducing variance and increasing generalization, especially when the number of variables per split was tuned. However, AdaBoost stood out for its ability to target hard-to-classify cases, even if it risked overfitting at lower learning rates.

I also realized how crucial it is to interpret not just the metrics, but why certain models perform better in context. Precision alone doesn't reflect the full story when predicting rare events like customer conversions — recall and PR-AUC give a better picture of the model's true utility. Ultimately, the most important part of this assignment wasn't the model scores — it was the process of learning how different algorithms think, and how tuning and preparation can completely change the narrative around a dataset.

Moving forward, I'd explore ensemble stacking to combine the strengths of each method, and possibly test other boosting algorithms like XGBoost or LightGBM. But more importantly, this project helped me move from just applying models to really questioning how data and design decisions affect outcomes — and that shift in mindset is what I'll carry into Assignment 3.

What's Next: Assignment 3 — Support Vector Machines (SVM)

In Assignment 3, I will explore the Support Vector Machine (SVM) classifier using an RBF kernel. This will introduce a new type of classifier — a discriminative model focused on identifying optimal decision boundaries.

The primary tuning parameters will include:

- **Cost (C):** Controls margin size and misclassification penalty.
- **RBF Sigma (σ):** Controls the nonlinearity of the decision boundary.

This experiment will complete the comparison across all major modeling strategies:

- Single classifier: Decision Tree
- Ensemble method: Random Forest
- Boosting method: AdaBoost
- Discriminative method: SVM

Let's see how well the SVM holds up against the top performer: Random Forest.

Previous Assignment: Assignment 1 – Bank Marketing Campaign EDA

Next Assignment: Assignment 3 – Support Vector Machines

Appendix

```
sessionInfo()
```

```
## R version 4.5.1 (2025-06-13 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] kableExtra_1.4.0   corrplot_0.95      PRROC_1.4          rlang_1.1.6
## [5] pROC_1.19.0.1      vip_0.4.1          themis_1.0.3       yardstick_1.3.2
## [9] workflowsets_1.1.1 workflows_1.3.0     tune_2.0.1         tailor_0.1.0
## [13] rsample_1.3.1      recipes_1.3.1      parsnip_1.3.3      modeldata_1.5.1
## [17] infer_1.0.9        dials_1.4.2        scales_1.4.0       broom_1.0.10
```

```
## [21] tidymodels_1.4.1    lubridate_1.9.4    forcats_1.0.0      stringr_1.5.2
## [25] dplyr_1.1.4         purrr_1.1.0        readr_2.1.5        tidyr_1.3.1
## [29] tibble_3.3.0        ggplot2_4.0.0      tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] magrittr_2.0.4      furrr_0.3.1         compiler_4.5.1
## [4] systemfonts_1.2.3  vctrs_0.6.5         lhs_1.2.0
## [7] pkgconfig_2.0.3     fastmap_1.2.0       backports_1.5.0
## [10] rmarkdown_2.29      prodlim_2025.04.28  tzdb_0.5.0
## [13] xfun_0.53           jsonlite_2.0.0      parallel_4.5.1
## [16] R6_2.6.1            stringi_1.8.7       RColorBrewer_1.1-3
## [19] ranger_0.17.0       parallelly_1.45.1   rpart_4.1.24
## [22] xgboost_1.7.11.1    Rcpp_1.1.0          iterators_1.0.14
## [25] knitr_1.50          future.apply_1.20.0 Matrix_1.7-4
## [28] splines_4.5.1       nnet_7.3-20         timechange_0.3.0
## [31] tidyselect_1.2.1    rstudioapi_0.17.1   yaml_2.3.10
## [34] timeDate_4041.110   codetools_0.2-20    listenr_0.9.1
## [37] lattice_0.22-7      withr_3.0.2         S7_0.2.0
## [40] evaluate_1.0.5      future_1.67.0       survival_3.8-3
## [43] xml2_1.4.0          pillar_1.11.1       foreach_1.5.2
## [46] generics_0.1.4      hms_1.1.3           globals_0.18.0
## [49] class_7.3-23        glue_1.8.0          ROSE_0.0-4
## [52] tools_4.5.1         data.table_1.17.8   gower_1.0.2
## [55] RANN_2.6.2          grid_4.5.1          ipred_0.9-15
## [58] cli_3.6.5           DiceDesign_1.10     textshaping_1.0.3
## [61] viridisLite_0.4.2   svglite_2.2.1       lava_1.8.1
## [64] gtable_0.3.6        GPfit_1.0-9         digest_0.6.37
## [67] farver_2.1.2        htmltools_0.5.8.1   lifecycle_1.0.4
## [70] hardhat_1.4.2       MASS_7.3-65         sparsevctrs_0.3.4
```